

CS143 Spring 2026 – Written Assignment 2

Due Monday, April 27, 2026 11:59 PM PDT

This assignment covers context free grammars and parsing. You may discuss this assignment with other students and work on the problems together. However, your write-up should be your own individual work, and you should indicate in your submission who you worked with, if applicable. Assignments can be submitted electronically through Gradescope as a PDF by 11:59 PM PDT. Please review the the course policies for more information: <https://web.stanford.edu/class/cs143/policies/>. A L^AT_EX template for writing your solutions is available on the course website. If you need to draw parse trees in L^AT_EX, you may use the `forest` package: <https://ctan.org/pkg/forest>.

1. Give a context-free grammar (CFG) for each of the following languages. Any grammar is acceptable—including ambiguous grammars—as long as it has the correct language. The start symbol should be S .

- (a) The set of all strings over the alphabet $\{2, -, +\}$ representing valid arithmetic expressions where each integer in the expression is a single digit and the expression evaluates to some value ≥ 0 . Note that the empty string ε is *not* in the language.

Example Strings in the Language:

$2+2$ $2-2+2$ $-2-2+2+2$

Strings not in the Language:

-2 $-2+22$ $2++2-2$

Solution:

- (b) The set of all strings over the alphabet $\{0, 1\}$ with exactly one more 0 than 1's.

Example Strings in the Language:

0 100 010 0010011

Strings not in the Language:

ε 00 01 0110 0100

Solution:

- (c) The set of all strings over the alphabet $\{0, 1\}$ in the language $L : \{0^i 1^j 0^k \mid j = i + k\}$.

Example Strings in the Language:

ε	10	01	0110	000111
---------------	----	----	------	--------

Strings not in the Language:

0	00	001	0010	01110
---	----	-----	------	-------

Solution:

- (d) The set of all strings over the alphabet $\{[,], \{, \}, , \}$ which are sets. We define a set to be a collection of zero or more comma-separated arrays enclosed in an open brace and a close brace. Similarly, we define an array to be a collection of zero or more comma-separated sets enclosed in an open bracket and a close bracket. **Note** that "," (comma) is in the alphabet.

Example Arrays:

$\{\{\}, \{\}\}$ $[\{\}]$ $[\{\{\}, \{\}\}]$

Example Sets:

$\{\{\}\}$ $\{\}$ $\{\{\{\}\}, \{\}\}$

Example Strings in the Language:

$\{\}$ $\{\{\}, [\{\{\}\}]\}$ $\{\{\{\}, \{\}, \{\}\}, [\{\}]\}$

Strings not in the Language:

$[\{\}]$ $\{\{\}\}$ $\{\{\{\}\}$

Solution:

2. Consider the following grammar for if-then-else expressions that involve a variable x :

$$E \rightarrow \text{if } x \text{ then } E \mid M$$

$$M \rightarrow \text{if } x \text{ then } M \text{ else } E \mid x$$

Is this grammar ambiguous or not? If yes, give an example of an expression with two different parse trees and draw the two parse trees. If not, explain why that is the case.

Solution:

3. (a) Left factor the following grammar:

$$S \rightarrow I \mid I - J \mid I + K$$

$$I \rightarrow (J - K) \mid (J)$$

$$J \rightarrow K1 \mid K2$$

$$K \rightarrow K3 \mid \varepsilon$$

Solution:

(b) Eliminate left recursion from the following grammar:

$$S \rightarrow STS \mid ST \mid T$$

$$T \rightarrow Ta \mid Tb \mid U$$

$$U \rightarrow T \mid c$$

Solution:

4. Consider the following CFG, where the set of terminals is $\{a, b, c, d, \#, ?\}$:

$$S \rightarrow \#UT \mid T?$$

$$T \rightarrow aS \mid bUc \mid \varepsilon$$

$$U \rightarrow aSc \mid bTd$$

- (a) Construct the FIRST sets for each of the nonterminals.

Solution:

- S :
- T :
- U :

- (b) Construct the FOLLOW sets for each of the nonterminals.

Solution:

- S :
- T :
- U :

- (c) Construct the LL(1) parsing table for the grammar.

Solution:

Nonterminal	a	b	c	d	$\#$	$?$	$\$$
S							
T							
U							

- (d) Show the sequence of stack, input and action configurations that occur during an LL(1) parse of the string “ $\#a?ca?$ ”. At the beginning of the parse, the stack should contain a single S .

Solution:

Stack	Input	Action

5. Consider the following grammar G over the alphabet $\{a, b, c\}$:

$$\begin{aligned}S' &\rightarrow S \\S &\rightarrow Aa \\S &\rightarrow Bb \\A &\rightarrow Ac \\A &\rightarrow \varepsilon \\B &\rightarrow Bc \\B &\rightarrow \varepsilon\end{aligned}$$

You want to implement G using an SLR(1) parser. Note that we have already added the $S' \rightarrow S$ production for you.

- (a) Construct the first state of the LR(0) machine, compute the FOLLOW sets of A and B, and point out the conflicts that prevent the grammar from being SLR(1).

Solution:

- (b) Show modifications to production 4 ($A \rightarrow Ac$) and production 6 ($B \rightarrow Bc$) to make the grammar SLR(1) while having the same language as the original grammar G . Explain the intuition behind this result.

Solution: